COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S3
DATE PERFORMED:	9/08/2026	**DATE SUBMITTED:**	9/09/2026
NAME:	Dimaguiba, Lorenzo James M.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HANDS-ON ACTIVITY NO. 7.1 **SORTING ALGORITHMS: BUBBLE, SELECTION, AND INSERTION SORT**

1. PROCEDURE OUTPUT

MAIN CODE

Dimaguiba_Sorting > main.cpp > ...

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "sorting.h"
const int maxSize = 100;

int main(){

    //ARRAY CREATION
    int dataset[maxSize];
    srand(time(0));

    for(int i = 0; i <maxSize; i++){
        dataset[i] = rand() % 5 + 1; //to limit size
    }

    //DISPLAY
    std::cout<<"Current Data Set: ";

    for(int i = 0; i < maxSize; i++){
        std::cout<<dataset[i]<<" ";
    }

    //Sorting Alorithm Functions:
    //Remove (//) to test which

    //bubbleSort(dataset, maxSize);

    //selectionSort(dataset, maxSize);

    //insertionSort(dataset, maxSize);

    std::cout<<"\nSorted Data Set: ";

    for(int i = 0; i < maxSize; i++){
        std::cout<<dataset[i]<<" ";
    }

    return 0;
}

```

HEADER

main.cpp | sorting.h | sortingSupp.h | supp.cpp

Dimaguiba_Sorting > sorting.h > ...

```cpp
#ifndef SORTING_H
#define SORTING_H
#include <iostream>

template <typename T>
void bubbleSort(T arr[], int maxSize){
    for (int i = 0; i < maxSize; i++){
        for(int j = i + 1; j < maxSize; j++){
            if(arr[i] > arr[j]){
                std::swap(arr[i], arr[j]);
            }
        }
    }
}

template <typename T>
int routine_smallest(T arr[], int k, int arrSize){
    int position = k; //PROBLEM FIXED MY GOODNESS eto lang pala problema

    T smallestElement = arr[k];

    for(int j = k + 1; j < arrSize; j++){
        if(arr[j] < smallestElement){
            smallestElement = arr[j];
            position = j;
        }
    }
    return position;
}

template <typename T>
void selectionSort(T arr[], int arrSize){
    int min;

    for(int i = 0; i < arrSize; i++){
        min = routine_smallest(arr, i, arrSize);
        std::swap(arr[i], arr[min]);
    }
}

template <typename T>
void insertionSort(T arr[], int arrSize){
    int k = 1, j;
    T temp;

    while (k < arrSize){
        temp = arr[k];
        j = k - 1;
        while(j >= 0 && temp <= arr[j]){
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = temp;
        k++;
    }
}

#endif
```

Code + Console Screenshot

```
Dimaguiba_Sorting > main.cpp > ...
1  #include <iostream>
2  #include <cstdlib>
3  #include <time.h>
4  #include "sorting.h"
5  const int maxSize = 100;
6
7  int main(){
8
9      //ARRAY CREATION
10     int dataset[maxSize];
11     srand(time(0));
12
13     for(int i = 0; i <maxSize; i++){
14         dataset[i] = rand() % 10 + 1; //to limit size
15     }
16
17     //DISPLAY
18     std::cout<<"Current Data Set: ";
19
20     for(int i = 0; i < maxSize; i++){
21         std::cout<<dataset[i]<<" ";
22     }
```

Console:

```
PS C:\Users\shosh\OneDrive\Documents\VSCODES> ${env:DEBUGINFOD_URLS}=''; & 'c:\Users\shosh\.vscode\extensions\ms-vscode.cpptools-1.33.8-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-jvdxirt-gfxy2exf.4es' '--stderr=Microsoft-MIEngine-Error-ojsuswfe.wwv' '--pid=Microsoft-MIEngine-Pid-djyq30jm.xjj' '--dbgExe=C:\msys64\ucrt64\bin\gdb.exe' '--interpreter=mi'
Current Data Set: 3 5 10 4 7 4 1 9 7 7 1 8 3 3 5 6 1 9 6 10 3 6 8 6 9 8 7 7 3 5 6 1 1 9 1 10 9 9 7 2 7 1 1 8 7 5 1 1 4 6 2 6 9 6 6 2 1 6 1 6 8 8 2 8 3 10 9 5 6 9 10 10 9 6 2 8 7 8 5 1 3 3 9 1 7 10 2 6 4 4 6 1 5 4 10 1 6 1 3 9
```

Observations

In order to create random values for creating an array, several libraries are utilized. The main libraries for this were the “cstdlib” and “time.h” libraries. Mainly for their “rand”, “srand” and “time” functions. “srand” is to set a seed with the current “time”. The seed functions as a root or sequence that the randomizer (rand) will follow. To limit the number size, a modulo operation was used to set the max number size. That finishes the first loop. The second loop displays the new array.

Table 7-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot

```
26
27      bubbleSort(dataset, maxSize);
```

Dimaguiba_Sorting > C sorting.h > bubbleSort<T>(T [], int)

```
1   #ifndef SORTING_H
2   #define SORTING_H
3   #include <iostream>
4
5   template <typename T>
6   void bubbleSort(T arr[], int maxSize){
7       for (int i = 0; i < maxSize; i++){
8           for(int j = i + 1; j < maxSize; j++){
9               if(arr[i] > arr[j]){
10                  std::swap(arr[i], arr[j]);
11              }
12          }
13      }
14  }
15
```

Console:

```
Current Data Set: 9 2 10 9 9 9 9 8 1 7 9 10 1 9 6 8 9 6 7 5 5 5 2 5 5 3 10 8 4 10 7 8 10 2 1 2 3 6 4 2 5 6 3 5 2 1
Sorted Data Set: 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5
```

Observations

The bubble sort as a sorting algorithm is quite simple. It makes use of nested loops. The first loop counts as the index number to be compared, then the second loop takes the element next to it (i + 1). Since it is sorting in an ascending order, it checks if the element before is greater than. If it is greater than the next element, a swap happens. This loop continues and several swaps occur till it is sorted.

Table 7-2. Bubble Sort Technique

Code + Console Screenshot

```
29      selectionSort(dataset, maxSize);
30
```

```
16  template <typename T>
17  int routine_smallest(T arr[], int k, int arrSize){
18      int position = k; //PROBLEM FIXED MY GOODNESS eto lang pala problema
19
20      T smallestElement = arr[k];
21
22      for(int j = k + 1; j < arrSize; j++){
23          if(arr[j] < smallestElement){
24              smallestElement = arr[j];
25              position = j;
26          }
27      }
28      return position;
29  }
30
31  template <typename T>
32  void selectionSort(T arr[], int arrSize){
33      int min;
34
35      for(int i = 0; i < arrSize; i++){
36          min = routine_smallest(arr, i, arrSize);
37          std::swap(arr[i], arr[min]);
38      }
39  }
40
```

Console:

```
Current Data Set: 5 8 5 10 10 7 3 3 9 9 4 4 8 10 5 9 2 4 5 6 5 4 2 6 9 1 7 5 6 5 6 6 1 4 2 9 2 7 4 6 3 1 3 8 10 6 4 1 1 4 4 2 1 5 6 6 3 1 2 6 2 2 5 1 5 1 4 4 3 2 4 10 5 8 7 3 8 10 3 9 1 2 10 8 9 8 4 10 2 10 3 8 10 5 4 8 3 1 1
Sorted Data Set: 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 6 6 6 6 6 6 6 6 6 6 6 7 7 7 7 7 8 8 8 8 8 8 8 8 8 9 9 9 9 9 9 9 9 10 10 10 10 10 10 10 10 10 10
```

Observations

Much like any other sorting algorithm, the result is the same. The method is what changed. The selection sort starts off by initializing the current element as the minimum element (being i). The first loop tracks the current index as steps going up the array. While the index of the minimum value is taken note of, the second loop begins or the (routine_smallest) function in which it begins comparing every element after the minimum element and if that element is greater than or less than that (depending on the condition) it sets the index of that element as the new min. If there are no more that meet that condition, a swap occurs. This continues on till sorted.

Table 7-3. Selection Sort Algorithm

Code + Console Screenshot

```
31      insertionSort(dataset, maxSize);
32
```

```
41  template <typename T>
42  void insertionSort(T arr[], int arrSize){
43      int k = 1, j;
44      T temp;
45
46      while (k < arrSize){
47          temp = arr[k];
48          j = k - 1;
49          while(j >= 0 && temp <= arr[j]){
50              arr[j + 1] = arr[j];
51              j--;
52          }
53          arr[j + 1] = temp;
54          k++;
55      }
56  }
57
58  #endif
```

Console:

```
Current Data Set: 5 3 10 5 9 8 10 8 9 2 4 6 6 2 1 3 1 7 1 6 9 10 2 1 5 8 8 6 7 1 10 3 5 8 1 9
Sorted Data Set: 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 4 4 4 4 4 4 5 5 5 5 5 5
```

Observations

The insertion sort takes a different approach that essentially works itself forward then backwards. It assumes the first element is already sorted. The first loop begins by assigning an element temporarily. “J” takes note of the element BEFORE the current element. Then begins the comparison stage. A condition of while the value less than the element before it. It begins to compare itself to elements before it. Once that condition breaks, a swap occurs. This goes on till the list is sorted.

Table 7-4. Insertion Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

CODE

Dimaguiba_Sorting > supp.cpp > main()

```cpp
#include <iostream>
#include <string>
#include <cstdlib>
#include <time.h>
#include "sortingSupp.h"

const int maxSize = 100;
const int canditateNum = 5;
int main(){
    int votes[maxSize];
    std::string candidates[canditateNum] = {"Bo Dalton Capistrano", "Cornelius Raymon Agustín", "Deja Jayla Bañaga", "Lalla Brielle Yabut", "Franklin Relano Castro"};
    //create random dataset
    srand(time(0));
    for(int i = 0; i < maxSize; i++){
        votes[i] = rand() % 5 + 1;
    }

    std::cout<<"Unsorted Votes: ";

    for(int i = 0; i < maxSize; i++){
        std::cout<<votes[i]<<" ";
    }

    sortSelect(votes, maxSize);
    std::cout<<"\nSorted Votes: ";
    for(int i = 0; i < maxSize; i++){
        std::cout<<votes[i]<<" ";
    }

    //vote count
    int win, voteNum, highest = 0;
    bool tied; //check ties
    for (int i = 0; i < canditateNum; i++){
        std::cout<<"\n"<<candidates[i];
        voteNum = instanceSearch(votes, i + 1, maxSize);
        if(voteNum > highest){
            highest = voteNum;
            win = i;
            tied = false;
        }

        else if(voteNum == highest){
            tied = true;
        }

    }

    if(tied){
        std::cout<<"\nVotes Tied!";}

    else{
        std::cout<<"\nWinner: "<<candidates[win];
    }


    return 0;
}
```

HEADER + MANUAL COUNT CODE

Dimaguiba_Sorting > C sortingSupp.h > routine_smallest<T>(T [], int, int)

```cpp
#ifndef SORTINGSUPP_H
#define SORTINGSUPP_H
#include <iostream>


template <typename T>
int routine_smallest(T arr[], int k, int arrSize){
    int pos = k;

    T smallest = arr[k];

    for(int j = k + 1; j < arrSize; j++){
        if(arr[j] < smallest){
            smallest = arr[j];
            pos = j;
        }
    }

    return pos;
}

template <typename T>
void sortSelect(T arr[], int arrSize){
    int min;

    for(int i = 0; i < arrSize; i++){
        min = routine_smallest(arr, i, arrSize);
        T temp = arr[i];
        arr[i] = arr[min];
        arr[min] = temp;
    }
}

//Linear search:
template <typename T>
int instanceSearch (T arr[], T search, int arrSize){
    int instances = 0;
    for(int i = 0; i < arrSize; i++){
        if(arr[i] == search){
            instances++;
        }
    }
    std::cout<<"\nNumber of votes: "<<instances<<std::endl;
    return instances;
}

#endif
```

OUTPUT + COUNT RESULT

```
t-0ylwe2tu.a2u' '--stderr=Microsoft-MIEngine-Error-jfqcckrr.jcs' '--pid=Microsoft-MIEngine-Pid-5oeiur2p.c2s' '--dbgExe=C:\msys64\ucrt64\bin\gdb.exe' '--interpreter=mi'
Unsorted Votes: 5 2 2 4 4 4 5 3 2 2 2 1 4 3 4 4 5 4 4 2 3 1 1 1 1 3 2 5 5 3 1 2 5 2 5 2 2 1 5 1 5 3 2 1 2 5 3 5 5 3 4 3 3 5 4 2 2 1 2 1 5 3 2 5 5 2 1 4 1 5 3 5 5 5 1 2 5 3 1 2 3 2 4 1 4 4 3 2 3 5 2 2 5 3 5 2 2 2 1 4
Sorted Votes: 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5
Bo Dalton Capistrano
Number of votes: 17

Cornelius Raymon Agustín
Number of votes: 27

Deja Jayla Bañaga
Number of votes: 17

Lalla Brielle Yabut
Number of votes: 15

Franklin Relano Castro
Number of votes: 24

Winner: Cornelius Raymon Agustín
```

IF TIED:

```
Votes Tied!
```

Question: Was your developed vote counting algorithm effective? Why or why not?

- I chose the selection sort algorithm due to it being more efficient. It also does the least amount of swaps compared to the other algorithms as it sorts through the entire list, only swapping after comparing every element forward. The vote counting algorithm was effective enough but I know that I can improve it further, but it does its job and it counts all the instances of every vote. I've made sure to also add a check; in an event of a tie.

Conclusion/Learnings:

Sorting algorithms are great tools used in ordering or rearranging elements to a certain order. The type of sorting algorithm also depends on the type of data being sorted and the total volume of it to be sorted. Be sure to keep note on the indexing on the loops and conditions. Make sure to properly initialize certain variables to avoid unpredictable behavior of your code.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The literal one line that stopped my selection sort algorithm from working correctly. Turns out "position" has to be initialized or else the indexing can be a mess that causes weird behavior.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.